

题目五：基于 UDP 的应用层可靠传输与 AI 驱动拥塞控制协议实现

背景介绍：

真实的 TCP 拥塞控制算法固化在操作系统的内核栈中，修改门槛极高。随着互联网的发展，谷歌等公司提出了基于 UDP 的 QUIC 协议，将可靠传输和拥塞控制移到了应用层。

本实验要求学生模拟 QUIC 的思想，基于纯 UDP Socket，在 Python 应用层从零实现一个带有“确认重传（ACK）”和“拥塞窗口（CWND）”机制的可靠传输协议。

学生将废弃传统的 AIMD（加性增乘性减）规则，引入强化学习 Q-Learning 算法，在线训练一个智能发送端。发送端通过实时监测应用层的 RTT 和丢包情况，利用 Q-Table 动态调整 UDP 的发送速率（CWND），最终在自己编写的协议上实现高吞吐、低延迟的传输。

实验目标：

1. 深入理解传输层协议内核，掌握基于原生 UDP 在应用层封装可靠传输机制；
2. 掌握窗口机制（CWND）、滑动队列与重传超时（RTO）的构建方法；
3. 实践网络状况的数字化表征（RTT 平滑采集、丢包率统计）；
4. 综合应用经典 AIMD 拥塞控制基线与前沿强化学习（如 Q-Learning）动态控制策略。

实验要求：

基础功能（必做）

1、应用层可靠传输协议建立：

自定义封包：设计 UDP 报文应用层，Sequence Number (4 字节) + Timestamp (8 字节) + Payload (1024 字节)。

确认与超时重传：Receiver 收到包后立刻回复携带 ACK Number 的 UDP 报文。Sender 需维护一个“未确认队列”，开启定时器，若超过设定的 RTO（重传超时时间）未收到 ACK，则判定为丢包并触发重传。

RTT 采样：Sender 收到 ACK 后，利用 当前时间 - 报文中的 Timestamp 计算出该数据包的 RTT，并平滑计算 SRTT。

2、虚拟瓶颈链路封装：

在 Sender 的 sendto() 外层构建模拟器。

模拟器设定一个固定带宽（如 100 包/秒，即每 10ms 漏出一个包发往真实网卡）和一个有限队列（如最大缓存 20 个包）。

当应用层瞬间把 CWND 大小的数据（例如 30 个包）塞给模拟器时：

产生真实延迟（排队）：前 20 个包进入虚拟队列。第 1 个包立刻发出，第 20 个包需要排队等待 $20 \times 10\text{ms} = 200\text{ms}$ 才会被真实 sendto() 发出。这模拟了拥塞导致的 RTT 飙升。

产生拥塞丢包（溢出）：后面的 10 个包因为虚拟队列已满（超出 20 的容量），被模拟器直接抛弃，不执行 sendto。这模拟了缓存溢出。

3、实现传统 AIMD 拥塞控制基线：

实现类似 TCP Reno 的规则作为对照组：

初始化 $\text{CWND} = 1$ （允许同时发送 1 个包，即未确认包数量上限）。

每收到一个 ACK: $\text{CWND} += 1 / \text{CWND}$ （拥塞避免：加性增）。

一旦发生超时丢包: $\text{CWND} = \max(1, \text{CWND} / 2)$ （拥塞惩罚：乘性减）。

记录该策略下的网络指标。

4、Q-Learning 智能拥塞控制器设计：

抛弃上述 AIMD 规则，利用 Q-Table 动态控制 CWND：

状态空间（State）：统计过去 1 个 RTT 内的网络特征，将其离散化为 6 个状态：RTT 趋势（变大/变小/平稳） $\times$ 丢包事件（发生/未发生）。

动作空间（Action）：每个 RTT 周期结束时，AI 决定下一个周期的行为：0 (CWND 保持), 1 (CWND + 1), 2 (CWND / 2)。

奖励函数（Reward）：设计一个复合奖励公式，例如： $R = \alpha \times \text{本轮成功吞吐量} - \beta \times \text{平均 RTT} - \gamma \times \text{丢包数量}$ 。让 AI 学会既要发得多，又不能让网络排队变卡。

在线学习：利用 ϵ -greedy 策略探索网络，并使用 Bellman 公式实时更新 Q-Table 矩阵。AI 可能无法在单次传输中收敛，要求灵活调整 ϵ 值，设计多轮循环训练机制。

5、数据记录与可视化：

程序运行结束后，使用 matplotlib 绘制对比图表：

CWND 随时间的变化曲线图（AIMD 的“锯齿波”与 Q-Learning 收敛后的“平滑波”）。
吞吐量与延迟柱状图（对比两种算法的平均 Mbps 和 平均 RTT）。

扩展功能

动态网络突变响应（10 分）：在传输到中途时，人为在代码中将 VirtualLink 的带宽突然减半。观察 AIMD 恢复耗时，及 Q-Learning 是否能遗忘旧策略快速收敛。

快速重传与乱序处理（15 分）：基础功能的 UDP 重传仅依赖超时，耗时极长。要求在应用层实现类似经典 TCP 的“连续收到 3 个重复 ACK 立即触发重传”机制，并正确处理乱序到达的数据包，提升网络利用率。

深度强化学习（引入 DQN/PPO 控制连续状态）（25 分）：废弃离散的 Q-Table，将精确的连续浮点数（当前精确 RTT、丢包率百分比、当前 CWND 大小）作为输入，利用 PyTorch 构建一个简单的神经网络模型，直接输出动作甚至连续的 CWND 乘数系数。探索神经网络在复杂网络状态下的控制潜力。

学生自定义开放性问题（10-25 分）：自己提出一个与“计算机网络架构”或“通信效率”相关的扩展问题，并给出代码实现方案。

技术栈要求：

建议在单台电脑上打开两个独立的终端窗口，分别作为独立进程运行 Sender.py 和 Receiver.py。双方基于本地环回地址（127.0.0.1）和不同端口进行 Socket 通信。

编程语言：Python 3。

网络核心库：socket（原生 UDP），threading（发送与接收并发处理），time（高精度时间戳与超时控制）。

数据计算：numpy（用于 Q-Table 矩阵运算与状态空间更新）。

可视化库：matplotlib（绘制性能对比图）。

分工建议：

- 可靠传输底层架构（Header 封装、定时重传、多线程控制）：1-2 人
- 虚拟网络拓扑模拟（漏斗模型与队列溢出模拟）：1 人

- 智能拥塞算法核心（AIMD 及 Q-Learning 状态/奖励函数设计）： 1 人
- 测试与可视化（Matplotlib 动态制图与数据比对）： 1 人

实验提交内容：

1. 代码

Sender.py 与 Receiver.py 及封装的虚拟链路类源码；

预训练的 Q-Table/模型权重文件；

2. 文档

设计文档：应用层 UDP 协议头图解、Q-Learning 奖励函数数学模型；

测试报告：Q-Learning 收敛曲线，AIMD 与 AI 算法吞吐量详细对比；

操作手册：代码执行顺序、环境包安装要求。

3. 演示

现场展示确认重传和拥塞控制功能的有效性；

实时展示并对比预训练的 Q-Learning 策略与 AIMD 策略下 CWND 演变的图表曲线。